

Sri Lanka Institute of Information Technology



IT3012 : Intelligent Agents – Lab 1

Y3.S1.WD.AI.01.01

Malabe Campus

Student ID -	IT24100140
Student Name -	Prathiban HTP
Module Name -	Intelligent Agents
Module Number -	IT3012
Lab Sheet Number -	1
Lab Sheet Date -	2026/July/24

Task 1

1.

P – Performance Measure

E - Environment

A - Actuators

S – Sensors

2.

```
def __init__(self, width=10, height=10, num_food=10, num_opponents=2, custom_walls=None):
    self.width = width
    self.height = height
    self.agent_pos = [0, 0] # Starting position (x, y)
```

Based on this code the init – width and height represents the grid size the whole game works and the number of food is the count that the agent can reach and the number of opponents is the one there will be other agents that hunts for food just as our agent, custom walls set to none means we don't going to spawn obstacles manually.

3.

```
# Generate adversarial opponents
self.opponents = []
while len(self.opponents) < num_opponents:
    ox = random.randint(0, self.width - 1)
    oy = random.randint(0, self.height - 1)
    op_pos = [ox, oy]
    if tuple(op_pos) != (0, 0) and tuple(op_pos) not in self.walls and tuple(op_pos) not in self.food_positions:
        self.opponents.append(op_pos)

self.score = 0
self.steps = 0
self.collison = False
```

Based on the code of the opponents there is one codes like which self.collison is false those are multi agentic behaviors to avoid walls and not collide with each other

4.

```
def get_percept(self) -> dict:
    return {
        'agent_pos': list(self.agent_pos),
        'opponent_positions': [list(op) for op in self.opponents],
        'smells_food': tuple(self.agent_pos) in self.food_positions,
        'hit_wall': tuple(self.agent_pos) in self.walls,
        'collision': self.collision,
        'score': self.score,
        'remaining_food': len(self.food_positions)
    }
```

A **percept sequence** in artificial intelligence is the complete chronological history of all sensory inputs **percepts** an intelligent agent has ever received from its environment.

In this code snippet, from the perception, the agent is trying to get all the sensory information which is (ex- agent_pos – his position right now, and the opponent position and many more)

5.

```
def get_percept(self) -> dict:
    return {
        'agent_pos': list(self.agent_pos),
        'opponent_positions': [list(op) for op in self.opponents],
        'smells_food': tuple(self.agent_pos) in self.food_positions,
        'hit_wall': tuple(self.agent_pos) in self.walls,
        'collision': self.collision,
        'score': self.score,
        'remaining_food': len(self.food_positions)
    }
```

S – Which is sensors

6.

Partially Observable – because the sensory data doesn't directly say where the food is, it only giving a hint like smells_food function

7.

```
def execute_action(self, action: str):
    self.steps += 1
    new_pos = list(self.agent_pos)

    if action == 'Up':
        new_pos[1] = min(self.height - 1, new_pos[1] + 1)
    elif action == 'Down':
        new_pos[1] = max(0, new_pos[1] - 1)
    elif action == 'Left':
        new_pos[0] = max(0, new_pos[0] - 1)
    elif action == 'Right':
        new_pos[0] = min(self.width - 1, new_pos[0] + 1)

    if tuple(new_pos) in self.walls:
        self.score -= 5
    else:
        self.agent_pos = new_pos

    tuple_pos = tuple(self.agent_pos)
    if tuple_pos in self.food_positions:
        self.food_positions.remove(tuple_pos)
        self.score += 20

    for op in self.opponents:
        move = random.choice(['Up', 'Down', 'Left', 'Right', 'Stay'])
        if move == 'Up' and op[1] < self.height - 1:
            op[1] += 1
        elif move == 'Down' and op[1] > 0:
            op[1] -= 1
        elif move == 'Left' and op[0] > 0:
            op[0] -= 1
        elif move == 'Right' and op[0] < self.width - 1:
            op[0] += 1

    if op == self.agent_pos:
        self.score -= 10
        self.collision = True

def is_done(self) -> bool:
    return len(self.food_positions) == 0 or self.steps >= 40 or self.collision
```

P – Performance Matrices

8.

To be able to make
as a security perspective,

Prevent exploits

Part 2

9.

It will limit agents ability to see the traps and he might fall onto those without thinking

10.

In the last question completely not having ability to see the traps is bad than having the ability to at least not to pinpoint but to at least approximately map the positions of the toxins to avoid it, it actually improves the rationality drastically

11.

Metric exploit happens when the agent finds a loop hole in the system and try to cheese the system and doing works to exploit the system instead of playing within the given rules